

AI 自動辨識技術導入 ERP 企業系統之挑戰與 實務論文之探討

作業編號：01
學生姓名：許允誠
系級：財金三
學號：112408540

目錄

壹、	問題敘述.....	3
貳、	目標.....	4
	2.1 核心思辨：從「被動顧問」到「受控代理人」的演進.....	4
	2.2 企業級 AI 落地的四大指標（引用並整合 Sarferaz 之 20 項核心需求）.....	4
參、	理論方法與業界實務實作探討.....	5
	3.1 業界實證結果與文獻結論.....	5
肆、	結論與心得.....	6
	4.1 結論：跨越理論與實務的鴻溝.....	6
	4.2 心得與反思：AI 時代的風險轉嫁與企業治理.....	6
伍、	參考文獻.....	8
陸、	附件(B 小題作業).....	8
	6-1 轉化函數之實作程式碼與圖片(使用 GOOGLE COLAB 完成).....	8

壹、 問題敘述

隨著人工智慧 (AI) 技術的爆發性成長, AI 已從單純的消費端應用, 全面滲透至企業的核心營運。身為中央大學管理學院的學生, 在修習財務金融與管理學科的過程中我深刻體認到: 真正的商業挑戰不在於如何使用 AI 聊天機器人, 而是如何將 AI 核心技術安全且有效地導入企業後台的大型 ERP (企業資源規劃) 系統。在進行財務報表分析或營運決策時, AI 究竟應定位為提供建議的「顧問」, 還是具備實質執行力的「代理人」? 這是我撰寫本報告的核心動機。

探究此問題前, 必須先釐清 AI 的技術層次。根據課堂講義, 技術核心由廣至深可分為三層: 人工智慧 (AI) 賦予機器感知與推理的能力; 機器學習 (ML) 提供透過經驗數據持續優化效能的方法; 而深度學習 (DL) 則利用多層次類神經網路進行複雜特徵的擷取。

機器學習近年之所以能取得突破性進展, 關鍵在於課堂所探討的「機器學習三角理論」。一個強大的 AI 模型, 必須仰賴三大基石的完美結合:

1. 資料 (Data): 機器學習的燃料。模型需透過海量且標記良好的歷史數據 (如過往財報、客訴信件) 進行訓練, 藉此提取特徵 (Features)。
2. 演算法 (Algorithm): 處理燃料的引擎。透過數學模型與類神經網路 (如卷積神經網路 CNN 或大型語言模型 LLM), 從資料中找出隱含規則與決策邊界 (Decision Boundary)。
3. 算力 (Computing Power): 驅動引擎的基礎設施。仰賴 GPU 或雲端運算中心的平行運算能力, 支撐龐大的矩陣計算。

然而, 當我們試圖將此「黃金三角」搬進高度要求準確與合規的企業 ERP 系統時, 現實的商業環境便浮現了嚴峻的挑戰:

1. 「資料」的隱私防線: ERP 系統內充斥著高度機密的財務數據與客戶個資 (PII)。企業如何在不洩露商業機密的前提下, 將資料餵給 AI 進行學習與辨識?
2. 「演算法」的黑盒子風險: 機器學習本質上是「機率運算」, 帶有非決定性 (Non-deterministic) 與幻覺風險。但財會流程要求 100% 的精確度與可解釋性, 企業該如何信任演算法做出的帳務決策?

3. 「算力」難題：在地端建置龐大算力成本過高，但將核心業務邏輯全數放上公有雲，又面臨延遲與資安威脅，系統架構該如何取捨？

基於上述動機，本報告將從此三大維度出發，並參考 SAP 系統的實際案例探討 AI 自動辨識技術導入 ERP 系統的實務痛點，解析現代企業如何跨越技術與實務的鴻溝。

貳、 目標

我們將跳脫單純的技術視角，從管理學與資訊系統的雙重維度出發，釐清 AI 在企業營運中的精確定位，並具體化其落地標準。本報告之核心目標可分為以下兩大構面：

2.1 核心思辨：從「被動顧問」到「受控代理人」的演進

在消費端市場，AI 多半扮演「顧問」的角色，負責統整資訊、草擬文案並提供建議，但最終的執行動作仍需由人類完成。然而，企業在面對海量且需即時處理的 ERP 任務（如帳單爭議、財務對帳）時，單純的顧問已無法解決營運瓶頸，受控的 AI 代理人才是未來企業的需求。

2.2 企業級 AI 落地的四大指標（引用並整合 Sarferaz 之 20 項核心需求）

為了將上述的「受控代理人」安全地融入 ERP 系統，本報告引述 SAP 首席架構師 Sarferaz (2025) 於 IEEE 發表之研究。Sarferaz 在其論文中針對 Agentic AI 提出了 20 項嚴格的系統必須遵守事項 (Requirements)。本節將 Sarferaz 的技術規範與防護思維，提煉為企業導入 AI 系統時必須達成的至少四大具體目標：

1. 系統架構與業務整合 (System & Business Integration)
2. 具現化執行力與人機協作 (Actionable Execution & Human-in-the-loop)
3. 安全合規與稽核治理 (Security, Compliance & Auditability)
4. 可靠度與跨平台延展性 (Reliability & Cross-platform Scalability)

以下對論文引用皆以[1]替代

參、理論方法與業界實務實作探討

3.1 業界實證結果與文獻結論

為了驗證 AI 代理在企業 ERP 系統中的實際可行性，本報告首先探討 [1] 於 2025 年在 IEEE 發表的實作結果。該研究將其建構的 Agentic AI 框架實際導入 SAP 核心產品的「爭議管理系統 (Dispute Management)」，並得出以下具體的營運數據與結論：

1. 極致的營運效率提升

根據[1]的實證數據，處理 1,000 筆歷史爭議紀錄的時間，從傳統人工比對的 128 小時，大幅縮減至僅需 0.9 小時（效能提升逾 140 倍）。在單筆處理速度上，標準辨識任務可於 1,000 毫秒（1 秒）內完成；即便是複雜的多合約條款比對，亦僅需 3,100 毫秒。

2. 顯著的營運成本縮減

該研究指出，導入 AI 代理後，企業每次爭議處理的綜合營運成本降低了 67%。此外，透過地雲協作架構，單次 AI 推理的雲端 API 呼叫成本成功控制在 0.002 至 0.008 美元之間，展現了極高的規模化經濟效益。

3. 辨識準確率與「非決定性」的權衡

在自動分類辨識的表現上，AI 代理達到了 81.7% 的準確率。[1] 在論文中坦言，此數據雖略低於傳統基於死板規則 (Rule-based) 系統的 97.3%，但 AI 代理的絕對優勢在於它能靈活解析傳統系統無法處理的「非結構化語意」（如情緒化的客戶抱怨信）。而那 10% 左右的誤差，正是為何系統必須內建「人類審核機制 (Human-in-the-loop)」的根本原因。

【文獻結論總結】

基於上述實證，[1]總結指出，結合「內容導向 (Content-based)」與「程式碼導向 (Code-based)」的混合代理架構，不僅在技術上完全可行，更成功橋接了理論 AI 與實務 ERP 落地之間的巨大鴻溝。作者認為，此一標準化的 AI 框架能為企業大幅降低技術導入門檻與員工培訓成本，並提供了堅實的經驗證據 (Empirical evidence)，證明 AI 代理已正式具備承接企業「任務關鍵型 (Mission-critical)」商業流程的能力。

肆、 結論與心得

4.1 結論：跨越理論與實務的鴻溝

綜合本報告之理論探討與實務案例分析，我們可以得出明確結論：AI 自動辨識技術導入 ERP 系統，已成功跨越了從「實驗室」到「商業實戰」的鴻溝。

回顧第一部分探討的「機器學習黃金三角（資料、演算法、算力）」，我們看見企業如何透過地雲協作的混合架構與編排服務，成功克服了 ERP 系統在資安、隱私防線與算力成本上的嚴峻挑戰。如同第三部分 [1] 的實證數據所示，AI 代理能為企業帶來 67% 的成本縮減與 140 倍的極致效率。

這解答了最初的提問：在現代企業的高壓營運環境中，AI 的定位已不再是紙上談兵的「顧問」，而是具備實質行動力、能接管任務關鍵型流程的「代理人」。然而，考量到 AI 演算法本身的非決定性（如 81.7% 的辨識率尚無法達到完美），企業必須建立起包含「知識圖譜」、「人類審核機制」與「不可竄改稽核軌跡」的安全防護網，才能真正實現我們所追求的終極目標——「受監管的代理人」。

4.2 心得與反思：AI 時代的風險轉嫁與企業治理

身為中央大學管理學院的學生，這次的實作分析與文獻研讀帶給我極大的震撼。過去，我們多半將焦點放在 AI 如何提升「個人」的生產力；但當 AI 代理人真正深入企業 ERP 系統的神經中樞時，我看到的是整個企業權力與風險結構的劇烈重組。

我深刻體認到，若未來企業全面施行 AI 代理人，將引發一個極需深思的管理學議題：「營運風險的極端轉嫁」。傳統的企業組織中，帳務辨識或合約處理的失誤風險，是分散在龐大的基層行政與審核人員身上的。然而，在全面自動化的未來，一旦 AI 代理的底層邏輯設定偏差，或是遭遇惡意的提示詞攻擊（Prompt Injection），其造成的連鎖財務損失與合規災難將瞬間爆發。

這意味著，巨大的企業風險將被高度集中轉嫁到兩類人上：

1. 系統端技術人員（IT 與架構師）：他們負責撰寫 AI 的業務邏輯與防護網，一旦設定出現漏洞，將直接導致系統性的營運危機。
2. 少數具備最終決策權的主管：在「Human-in-the-loop」機制下，主管成為處理 AI 異常判斷的最後一道防線。他們將面臨極高的決策壓力，甚

至可能因為過度依賴 AI 判斷而產生偏見。

總而言之，未來的管理人才不能再將 AI 視為單純的資訊工具，而必須將其視為高風險、高產出的「數位員工」。我們未來需要具備的核心能力，不再是手動撈取報表，而是如何制定 AI 的授權邊界、解讀系統稽核軌跡，以及在效率與風險之間取得精準平衡。這不僅是技術的演進，更是未來企業內部控制與公司治理必須嚴陣以待的核心挑戰。

伍、 參考文獻

- [1] Sarferaz, S. (2025). A Framework for Implementing Agentic AI Applications in Enterprise Resource Planning (ERP) Systems. IEEE Access, Vol. 13.
- [2] 李俊賢。《人工智慧簡論 (AI an overview_v2)》 [課堂講義]。國立中央大學資訊管理學系。
- [3] 李俊賢。《類神經網路基礎 (Basics of Artificial Neural Networks)》 [課堂講義]。國立中央大學資訊管理學系。
- [4] SAP 企業資源規劃系統實作資料：Figure 10. Dispute Management - AI Agent Conversation (SAP Joule AI 代理處理帳單爭議之實作介面)。

陸、 附件(B 小題作業)

6-1 轉化函數之實作程式碼與圖片(使用 GOOGLE COLAB 完成)

```
import numpy as np

import matplotlib.pyplot as plt

#建立橫軸 n，從 -5 到 5，切 1000 個點（讓曲線平滑）

n = np.linspace(-5, 5, 1000)

# 定義輸入向量 n（橫軸範圍）

n = np.linspace(-5, 5, 1000)


# 1. Hard Limit

#常用於分類決策的輸出層，將連續輸入轉化為二進位輸出。

#當 n<0 時 a=0，當 n>=0 時 a=1。

a_hardlim = np.where(n < 0, 0, 1)
```

2. Symmetrical Hard Limit

#當 $n < 0$ 時 $a = -1$ ，當 $n \geq 0$ 時， $a = 1$ 。

#與 Hard Limit 相似，但輸出範圍在 -1 與 1 之間，適用於雙極性分類。

```
a_hardlims = np.where(n < 0, -1, 1)
```

3. Linear

$a = n$

#線性轉換，通常用於神經網路的線性逼近或輸出層

```
a_purelin = n
```

4. Saturating Linear

#在 $[0, 1]$ 區間內為線性，小於 0 則為 0 ，大於 1 則為 1 。

#結合了線性與硬限制的特質，具備飽和區間

```
a_satlin = np.pieceswise(n, [n < 0, (n >= 0) & (n <= 1), n > 1], [0,  
lambda n: n, 1])
```

5. Symmetric Saturating Linear

#在 $[-1, 1]$ 區間內為線性，其餘部分則飽和於 -1 或 1 。

```
a_satlins = np.pieceswise(n, [n < -1, (n >= -1) & (n <= 1), n > 1], [-  
1, lambda n: n, 1])
```

6. Log-Sigmoid

$a = 1 / (1 + e^{-n})$

#輸出範圍在 $(0, 1)$ 之間。

```
a_logsig = 1 / (1 + np.exp(-n))
```

```
# 7. Hyperbolic Tangent Sigmoid
```

```
#a=(e^n-e^-n)/(e^n+e^-n)
```

```
#輸出範圍在(-1, 1)，其數學性質較 logsig 更具對稱性，有利於模型收斂。
```

```
a_tansig = (np.exp(n) - np.exp(-n)) / (np.exp(n) + np.exp(-n))
```

```
# 8. Positive Linear
```

```
#當 n<0 時 a=0；當 n>=0 時 a=n。
```

```
#在深度學習中稱為 ReLU，能有效解決梯度消失問題
```

```
a_poslin = np.where(n < 0, 0, n)
```

```
titles = ['Hard Limit', 'Symmetrical Hard Limit', 'Linear',  
'Saturating Linear',
```

```
          'Symmetric Saturating Linear', 'Log-Sigmoid', 'Hyperbolic  
Tangent Sigmoid', 'Positive Linear']
```

```
data = [a_hardlim, a_hardlims, a_purelin, a_satlin, a_satlins,  
a_logsig, a_tansig, a_poslin]
```

```
plt.figure(figsize=(16, 12))
```

```
for i in range(8):
```

```
    plt.subplot(4, 2, i+1)
```

```
    plt.plot(n, data[i], 'b', linewidth=2)
```

```
    plt.title(titles[i])
```

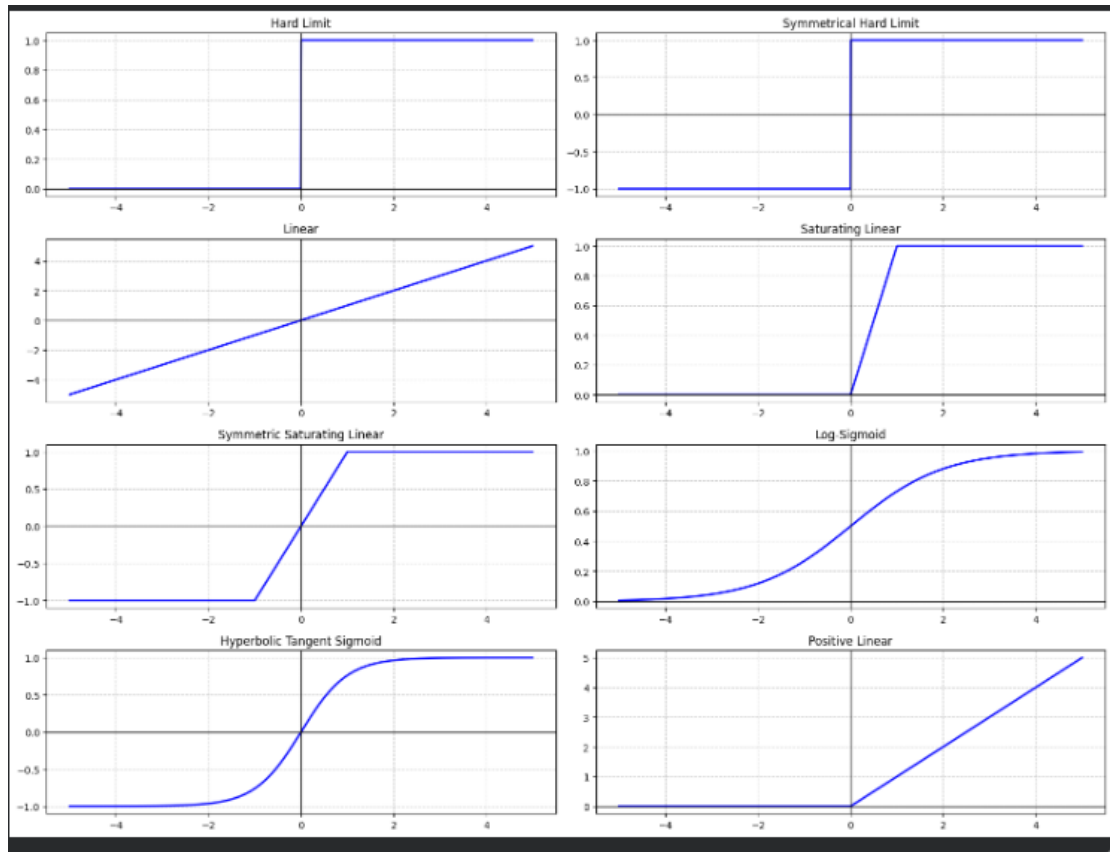
```
    plt.axhline(0, color='black', lw=1)
```

```
    plt.axvline(0, color='black', lw=1)
```

```
plt.grid(True, linestyle='--', alpha=0.7)
```

```
plt.tight_layout()
```

```
plt.show()
```



(另有.pynb 檔案附註)